

Deep Residual Learning for Image Recognition

<https://arxiv.org/pdf/1512.03385>

▼ 0. Abstract

깊은 신경망 학습을 위한 residual learning framework 제안.

explicitly **reformulate the layers**

- as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions.

장점

- easier to optimize,
- and can gain accuracy from considerably increased depth.

실험 결과

- On the ImageNet dataset
 - evaluate residual nets with a depth of up to 152 layers (8× deeper than VGG nets)
 - but still having lower complexity.
 - An ensemble of these residual nets achieves 3.57% error on the ImageNet test set.
 - This result won the 1st place on the ILSVRC 2015 classification task.
 - We also present analysis on CIFAR-10 with 100 and 1000 layers.
 - The depth of representations is of central importance for many visual recognition tasks.
 - Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset.
- Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions¹, where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation

▼ 1. Introduction

Driven by the significance of depth, a question arises:

Is learning better networks as easy as stacking more layers?

⇒ vanishing/exploding gradients로 인해 답하기 어려운 문제였지만

- normalized initialization
- intermediate normalization layers

enable networks with tens of layers to start converging for stochastic gradient descent (SGD) with backpropagation

⇒ BUT **degradation problem(degradation of training accuracy)**

Layer가 깊어질수록 **accuracy gets saturated** and then **degrades rapidly**.

이론적으로 이미 학습된 shallow network에 층을 추가하여 더 깊은 신경망을 만든다고 가정하면

추가된 층들이 단순히 입력값을 그대로 출력하는 Identity Mapping 역할을 수행하고, 이전 층들의 가중치를 그대로 복사해 온다면, 최소한 기존의 얇은 모델과 동일 성능을 내야한다. (구조적으로 training error이 더 높아질 이유가 없음)

실제 현상(Degradation): 오히려 단순히 층을 깊게 쌓은 신경망(plain network)은 얇은 신경망보다 학습 오차가 더 높음

- 모델 자체가 나빠서가 아니라,
- 깊은 신경망의 가중치를 **identity mapping으로 최적화하는 것이 매우 어렵기 때문**. (Optimization 방식인 SGD가 Identity Mapping을 찾아내도록 신경망의 가중치를 학습시키는 데 실패하고 있음을 의미.)
- 즉, 이론적으로 더 좋은 solution을 가짐에도 불구하고 실제 Optimization 과정에선 이걸 효율적으로 찾아내지 못함

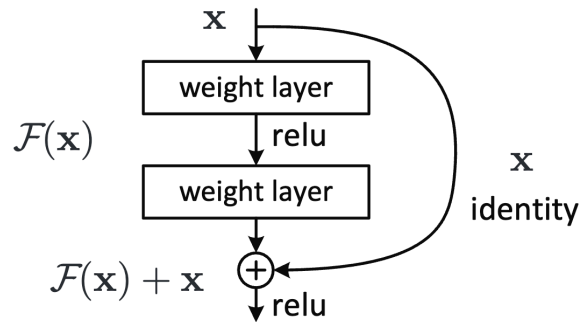


Figure 2. Residual learning: a building block.

⇒ introduce **a deep residual learning framework.**

- 기존 방식: 쌓인 레이어들이 직접 원하는 출력값인 $H(x)$ 을 찾아내도록 학습
- 제안 방식: 레이어들이 학습해야할 대상을 $H(x)$ 가 아니라, 입력값 x 와의 차이인 residual $F(x) := H(x) - x$ 로 바꾼다.
- 이후 최종 출력은 다시 $F(x) + x$ 로 재구성하여 원래의 매핑 $H(x)$ 을 얻는다. ⇒ Shortcut Connection / Skip Connection 이라고 부른다.
- hypothesize that it is easier to optimize the residual mapping than to optimize the original(unreferenced mapping)
- 만약 특정 레이어들이 학습해야할 optimal한 mapping이 identity mapping(입력을 그대로 전달하는 것)이라면,
 - 기존 신경망: 복잡한 비선형 레이어들을 통해 항등 함수를 근사해야함
 - Residual Learning Framework: 레이어들의 가중치를 0으로 수렴시키는 것만으로 쉽게 $F(x) \approx 0$ 을 만들어 항등 함수 구현 가능 ⇒ 더 쉬운 최적화

▼ 2. Related Works

Residual Representations

기준점과의 차이를 표현하는 게 효과적이라는 관찰:

- VLAD, 벡터 양자화, PDE의 multigrid방법 등의 사례에서 reformulation과 preconditioning을 통해 최적화 문제를 훨씬 쉽게 만든다는 시사점 제공.

Shortcut Connections

- 초기 MLP에선 네트워크 입력에서 출력까지 바로 연결되는 linear layer를 추가하는 방식이 오래 전부터 있었음.
- GoogLeNet에서 auxiliary classifier: 중간 층들을 별도의 보조 classifier에 직접 연결하여 vanishing/exploding gradient 문제에 대응
- Layer response centering 기법들

Highway Networks와의 결정적 차이

동시대 연구인 **Highway Networks**가 shortcut connection을 쓴다는 점에서 가장 유사한 경쟁 연구인데, 여기서 핵심적인 차이를 짚어:

	Highway Networks	ResNet (이 논문)
Shortcut 종류	gated shortcut (데이터에 따라 열고 닫힘, 파라미터 있음)	identity shortcut (항상 열려있음, 파라미터 없음)
Gate가 닫히면	non-residual function을 표현 (잔차 학습이 아니게 됨)	해당 없음 — shortcut이 절대 안 닫힘
정보 흐름	gate 상태에 따라 정보가 막힐 수 있음	모든 정보가 항상 그대로 통과하고, 그 위에 residual function이 추가로 학습됨
깊이에 따른 성능	100층 이상의 극단적으로 깊은 모델에서 정확도 향상을 보여주지 못함	152층까지 정확도 이득을 명확히 입증

shortcut connection이라는 개념 자체는 새로운 게 아니지만 이 논문은

- (1) 파라미터가 전혀 없는 순수 identity shortcut을 쓰고,
- (2) shortcut이 항상 열려있어서 모든 정보가 항상 전달되고,
- (3) 이걸 통해 100층+의 깊이에서 실제 정확도 향상을 입증했다는 점에서 차별화함.

▼ 3. Deep Residual Learning

▼ 3.1. Residual Learning

$\mathcal{H}(\mathbf{x})$: underlying mapping to be fit by a few stacked layers

해당 stacked layers의 input = $\mathbf{x}$

$\mathbf{x}$: inputs to the first of these layers

가정: 여러 비선형 층이 점진적으로 복잡한 함수 $\mathcal{H}(\mathbf{x})$ 를 근사할 수 있다고 가정

⇒ residual function $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$ 을 근사할 수 있다고 가정하는 것과 동치.

이때 $\mathcal{F}(\mathbf{x})$ 을 학습하기 위해선 input $\mathbf{x}$ 와 output $\mathcal{H}(\mathbf{x})$ 을 원소별로 뺄 수 있어야하기 때문에 둘의 차원이 같아야한다.

$\mathcal{F}(\mathbf{x})$ 예측 후 나머지 $\mathbf{x}$ 를 덧붙인다면 original function을 예측하는 것이 될 것

그래서 Layer가 $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$ 을 근사하도록 만든다.

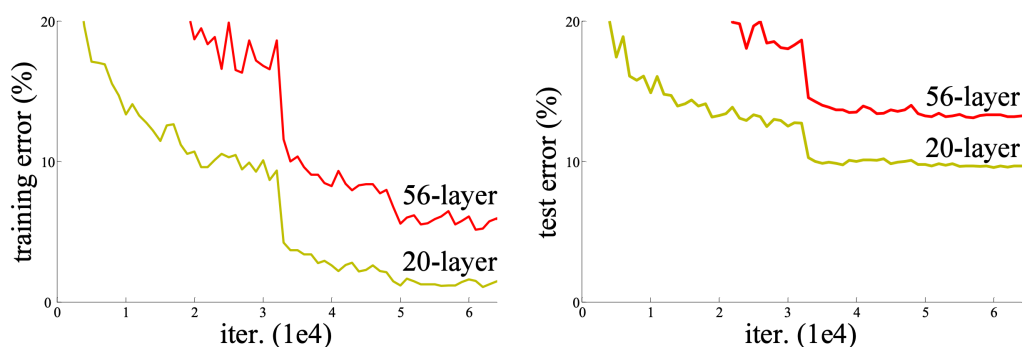


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

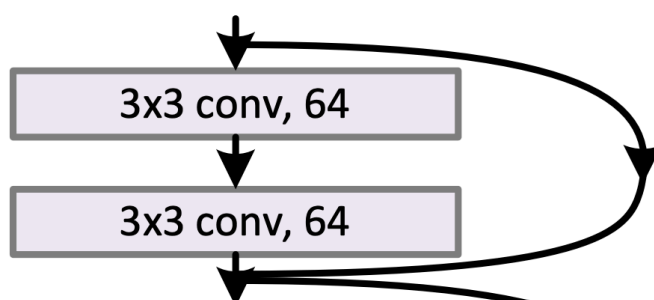
- 위 figure 1에서 알 수 있는 것은 degradation problem이 과적합으로 인한 문제는 아니라는 것.

▼ 3.2. Identity Mapping by Shortcuts

building block: $y = F(x, \{W_i\}) + x$

- x, y : input and output vectors of the layers
- function $F(x, \{W_i\})$: residual mapping to be learned
-

The operation $F + x$ is performed by a shortcut connection and element-wise addition



- **shortcut connection:** 블록에 들어오는 input x 을 아무연산없이 그대로 전달하는 것(=identity shortcut), 옆으로 우회해서 바로 출력 쪽으로 가는 화살표.
- **element-wise addition:** shortcut을 타고 온 x 와, 메인 경로(weight layers)를 거쳐 $F(x)$ 을 최종적으로 합치는 연산. 텐서의 같은 위치(같은 채널, 같은 공간 위치)끼리 값을 더하는 것 (행렬곱이나 concat이 아니라 그냥 같은 shape끼리 +). 그래서 **$F(x)$ 와 x 는 반드시 dimension이 같아야 함**

$F(x)$ 와 x 는 반드시 dimension이 같아야 함

input/output channel을 바꾸는 등 차원이 동일하지 않은 경우

여기서 차원은 공간 크기가 아니라 채널 수(filter수)를 의미함. stage가 바뀔 때 feature map 크기가 절반으로 줄고, 채널 수는 2배로 늘어난다. (dotted line shortcut)

→ 차원을 같게끔하기 위해 input x 에 linear projection W_s 을 수행하여 output과 차원을 맞추어준다.

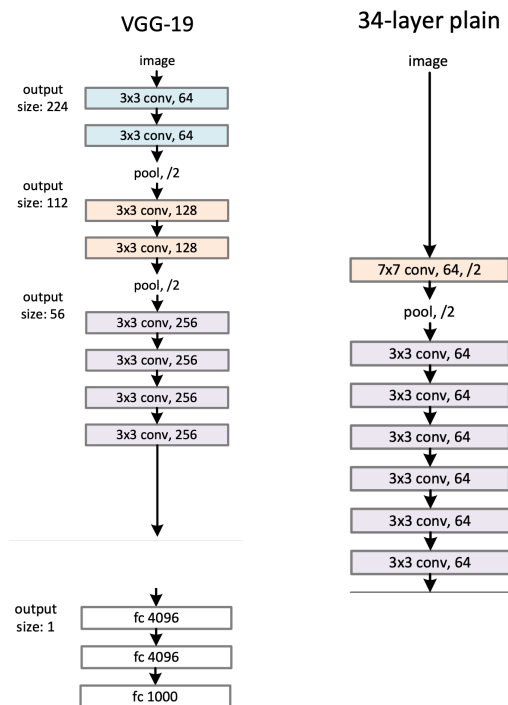
▼ 3.3. Network Architectures

Plain Network

- plain baseline inspired by VGGnets

- VGGnets: 깊은 네트워크를 쌓는 모델

- 3×3 conv filter만 사용 (더 큰 필터 대신 작은 필터를 여러 겹 쌓아서 큰 receptive field를 얻음)
- conv layer들을 쌓다가 중간 중간 pooling(max pooling, stride 2)으로 feature map 크기를 절반으로 줄임
- 맨 끝에 fully-connected layer 3개(fc4096, fc4096, fc1000)를 붙여서 최종 분류



- 같은 output feature map크기를 갖는 layer들은 같은 개수의 filter를 갖도록 - figure3에서 같은 output size를 갖는 구간(같은 색)의 conv들은 모두 filter개수가 동일
- feature map크기가 절반으로 줄어들면(downsampling) filter개수는 두 배로 늘림. layer당 연산량 유지를 위해(feature map이 작아진 만큼 채널을 늘려서 균형 맞추기 위함) 왜?
- downsampling: stride 2 conv layer자체로 수행. pooling layer가 각 stage 사이에 딱 하나만 있고 나머지는 conv layer가 직접 stride2로 크기를 줄임.
-

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

- VGG처럼 fc layer여러 개(4096,4096) 쌓지 않고 global average pooling 하나 + fc1000 하나만 사용. VGG의 파라미터 대부분이 fc layer에 몰려있는데 이걸 global average pooling으로 대체하면서 파라미터 수와 연산량이 줄어든다.
-

Residual Network

- plain network에서 shortcut connection 추가
 - identity shortcut : input output 같은 차원일 때 사용
 - 차원 input < output 일 때
 - (A) shortcut performs identity mapping with extra zero entries padded for increasing dimensions (추가 파라미터 없음) → residual learning이 기여하지 못함. 새로 생긴 채널들에 전달할 x 자체가 없으니 identity shortcut의 이점이 발휘되지 않는다.
 - (B) projection shortcut to match dimensions(by 1×1 conv.)
 1×1 convoluition은 공간 방향으로 아무것도 안보고 오직 채널만 섞어서 다른 채널 수로 변환하는 연산. 즉 선형 결합(가중합)해서 새로운 채널값으로 만들어내는 방식.

 원래 x의 64채널 정보를 학습 가능한 가중치 W_s 로 변환해서 128채널을 만들어내는 것. 대신 학습가능한 파라미터 W_s 를 추가로 필요로함.
 - 두 옵션 모두 stride=2

▼ 3.4. Implementation

1. data augmentation

- scale augmentation: 원본이미지를 그대로 쓰지 않고 짧은 변의 길이를 256~480픽셀 사이에서 무작위로 골라 resize.
- random cropping+horizontal flip: 위에서 리사이즈된 이미지에서 224x224크기의 영역을 무작위 위치에서 잘라냄. 그리고 그걸 좌우로 뒤집은 버전(horizontal flip)에서도 무작위로 자를 수 있도록. 어느 위치를 crop할지+뒤집을지 말지가 랜덤으로
- per-pixel mean subtracted: centering을 통해 학습 안정화를 위한 normalization?
- standard color augmentation: RGB색상값에 PCA를 적용해서 나온 주요 색상 변화 방향으로 랜덤한 색상 변형을 가하는 것. 조명 변환/색감 차이의 robustness를 위해

2. convolution → batch normalization → activation

BN: 미니배치 단위로 각 층의 출력값을 평균 0, 분산 1로 정규화한 다음 학습가 능한 scale/shift 파라미터로 재조정하는 기법. 입력 분포가 학습 도중 계속 바뀌는 문제에 대응하기 위함.

conv연산 결과는 학습 초기에 스케일이 들쭉날쭉하고 커질 수 있어서 바로 ReLU에 넣으면 학습 불안정해지므로 conv직후에 BN으로 정규화 후 ReLU에 넣어 안정적으로 학습

3. initialize the weights as in [13] and train all plain/residual nets from scratch

[13]: 저자들의 이전 논문(Delving Deep into Rectifiers)에서 제안한 초기화 방법(He initialization / MSRA initialization)은 ReLU를 쓰는 네트워크에 특화된 초기화 방식: ReLU가 절반의 입력을 0으로 만들어버리는 걸 고려해서 분산이 적절히 유지되도록 초기 가중치의 분산을 특정 공식으로 설정 → 초기 forward/backward신호가 너무 커지거나 작아지는 것 방지.

4. Dropout 안씀

Dropout은 학습 중 매 스텝마다 일부 뉴런을 무작위로 꺼서(0으로 만들어) 학습시키는 regularization 기법으로 뉴런들이 서로 과도하게 의존(co-adaptation)하는 걸 막아 overfitting을 줄인다.

BN자체가 어느정도 regularization 효과를 제공하기 때문에 dropout안씀?

5. Test: 10-crop testing

하나의 이미지에 대해 10개의 서로 다른 224x224 crop을 만들어서 각각 모델에 통과시키고 그 10개 예측 결과를 평균내서 최종 예측으로 사용하는 방법. (일종의 test-time augmentation/ensemble 효과)

6. fully-convolutional form as in [41,13] and average the scores at multiple scales:

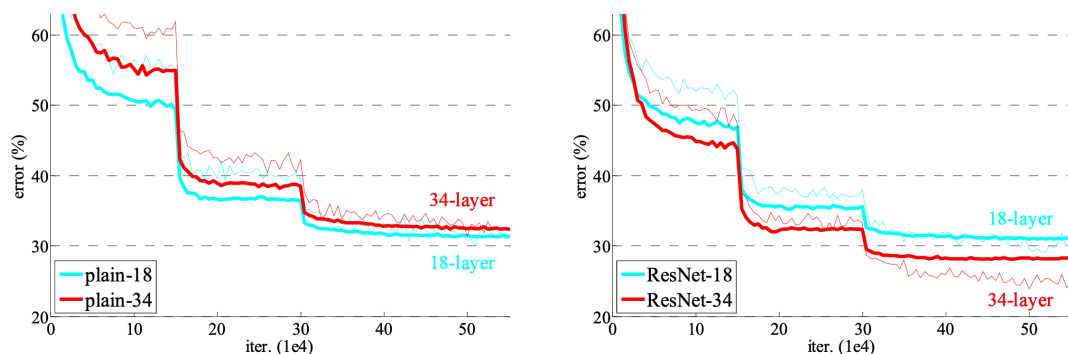
원래 마지막에 fc layer가 있으면 입력 이미지 크기가 고정되어야함 (224x224). fc layer를 1x1 conv처럼 동작하게 재해석하면 임의 크기의 입력 이미지를 통째로 받아서 처리 가능. crop을 여러번 잘라낼 필요없이 이미지 전체를 한번에 통과시켜서 여러 위치에 대한 예측 점수 맵을 한번에 얻는 방식.

fully-convolutional 네트워크에 원본 이미지를 여러 다른 크기로 리사이즈한 버전들을 각각 통과시켜서 여러 스케일에서 얻은 예측 점수들을 평균내서 최종 예측으로 사용.

5,6 중 최고 성능을 위해서는 6을 사용.

▼ 4. Experiments

▼ 4.1. ImageNet Classification



thin curve=training error, bold curve=validation error

▼ Plain Networks

- 보통 validation error > training error라면 overfitting을 의심하는데, 위 결과에선 18, 34 layer에서 모두 training error가 높게 나옴 → 애초에 최적화 자체가 잘 안되고 있음을 의미 → degradation problem
 - vanishing gradient가 원인이 아니다
 - BN을 쓰고 있어서 forward로 전달되는 신호가 0으로 죽지않고 non-zero variance을 유지하고 있음을 확인
 - backward로 전달되는 gradient의 norm도 healthy(너무 작아지지 않음)
 - 깊은 plain network의 solution space가 너무 복잡해서 SGD가 optimum(좋은 해)에 도달하는 데 필요한 시간(convergence rate)이 네트워크가 깊어질수록 exponentially 느려질 수 있다
 - iteration을 3배로 늘려서 더 오래 학습시켜봤음에도 불구하고 여전히 degradation 나타남. optimization자체가 근본적으로 어려워진 상태.
- figure에서도 training error가 plain-18 < plain-34 이다.

▼ Residual Networks

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Top-1 error (% , 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts

- 34-layer ResNet better than 18-layer ResNet: 깊어질수록 성능 좋아짐
- 34-layer ResNet has lower training error and generalizable to validation data: 만약 training error는 비슷한데 validation만 좋아진거라면 regularization역할을 해서 overfitting을 줄여준 것으로 해석가능한데, 실험 결과 training error도 낮췄고 이게 validation error개선으로 이어졌으므로 최적화 문제 자체를 풀어준 것으로 해석 가능하다.

⇒ degradation problem well addressed & manage to obtain accuracy gains from increased depth.

compared to plain network,

- 34-layer ResNet reduces the top-1 error by 3.5% ← shortcut을 추가하는 것 하나만으로 top-1 error가 3.5%p 줄어들음, effectiveness of residual learning on extremely deep systems.

18-layer plain/residual nets are comparably accurate. but the 18-layer ResNet converges faster

- 그렇게 네트워크가 심하게 깊지 않을 때 SGD slover가 plain net에서 어느 정도 잘 optimize.
- ResNet으로는 faster convergence

▼ Identity vs. Projection Shortcuts

세가지를 비교:

- A: zero-padding shortcuts for increasing dimensions
- B: projection shortcuts for increasing dimensions + 나머지는 identity
- C: all projection shortcuts

셋 다 plain network보다는 better

B slightly better than A

- b/c zero padded dimensions in A have no residual learning

C slightly better than B

- b/c extra parameters by many projection shortcuts

BUT small difference between A/B/C indicate that projection shortcuts are not essential for addressing the degradation problem ← not use C for rest of the paper

Identity shortcuts are particularly important for not increasing 복잡도 of 병목이 되는 아키텍처 (하단 언급)

▼ Deeper Bottleneck Architectures

bottleneck design: for each residual function F , use a stack of 3 layers instead of 2.

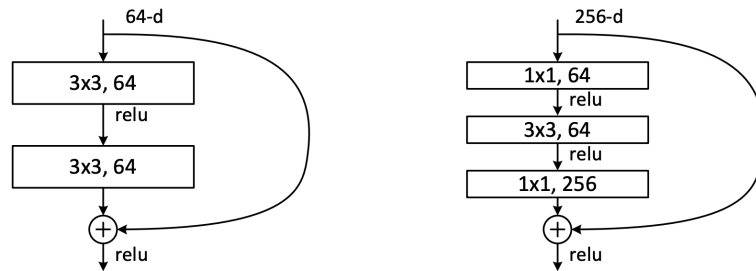


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

3×3 layer has a bottleneck with smaller input/output dimensions

parameter-free identity shortcuts 특히 중요 for the bottleneck 아키텍처

- if identity shortcut이 projection shortcut으로 대체된다면,
shortcut이 두 고차원 지점에 연결되었기 때문에 시간복잡도와 모델사이즈가 두배가됨을 보일 수 있다.

replace each 2-layer block in the 34-layer net with this 3-layer bottleneck block

→ **50-layer ResNet**

using more 3-layer blocks, construct 101 layer and 152 layer ResNets

- 비록 깊이는 훨씬 증가했음에도, 152-ResNet has lower complexity than VGG-16/19 nets
- 50/101/152-layer ResNets are more accurate than the 34-layer ones

degradation problem이 관측되지 않는다 + 상당한 accuracy gain from increased depth

▼ Comparisons with SOTA Methods

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except [†] reported on the test set).

method	top-5 err. (test)
VGG [41] (ILSVRC'14)	7.32
GoogLeNet [44] (ILSVRC'14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	3.57

Table 5. Error rates (%) of **ensembles**. The top-5 error is on the test set of ImageNet and reported by the test server.

ResNet 아키텍처를 depth를 다르게 해서 여러 개(6개) 따로따로 학습시킨 다음 그 6개 모델의 예측 확률 (softmax output)을 평균 내서 최종 예측으로 쓴 앙상블

4.2. CIFAR-10 and Analysis

- 아주 단순한 아키텍처로 극단적으로 깊은 네트워크에서의 behavior을 관찰하고자 함.
- 데이터셋은 CIFAR-10 (5만 장 training, 1만 장 test, 10개 class, 32×32 작은 이미지).

핵심 결론

1. Degradation 문제와 residual learning의 효과는 ImageNet만의 특수한 현상이 아니라 CIFAR-10에서도 동일하게 재현됨 → 일반적인 현상
2. Layer response 분석은 residual function이 0에 가깝게 학습된다는 저자들의 가설을 직접적으로 뒷받침
3. 1000층 이상에서도 optimization(학습) 자체는 문제없이 잘 되지만, 그렇다고 무조건 성능이 좋아지는 건 아니고 이제는 overfitting이라는 별개의 문제가 등장함 (깊이를 늘리는 것과 일반화 성능을 높이는 것은 서로 다른 문제)

네트워크 구조: ImageNet을 그대로 따르되 훨씬 작게 설계:

- 입력: 32×32 이미지

- 첫 layer: 3×3 conv
- 이후 $6n$ 개의 layer를 쌓는데 feature map 크기 $\{32, 16, 8\}$ 각각에 대해 $2n$ 개 layer씩 배정 (크기가 줄어든 때마다 채널 수는 $\{16, 32, 64\}$ 로 늘어남)
- 끝은 GAP + 10-way fc + softmax
- 총 층수 = $6n+2$
- Shortcut을 쓸 경우 $3n$ 개의 shortcut이 생기고(2층씩 묶은 block마다 하나), 모두 identity shortcut(option A)만 사용 (순수하게 shortcut 유무만 비교)

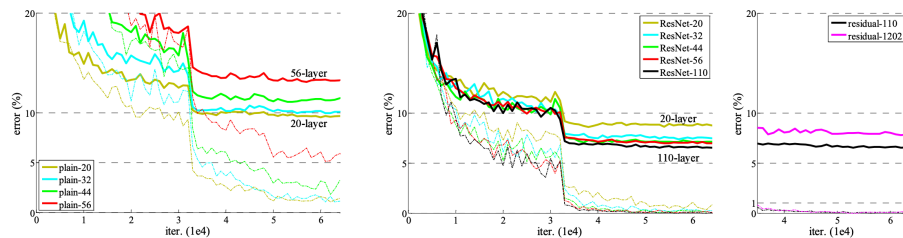


Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left**: plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle**: ResNets. **Right**: ResNets with 110 and 1202 layers.

실험 1: $n = \{3, 5, 7, 9\} \rightarrow 20/32/44/56$ 층 비교 (Fig. 6 왼쪽/가운데)

- **Plain nets**: 층이 깊어질수록(20 $\rightarrow$ 56층) training error가 오히려 증가 $\rightarrow$ ImageNet과 동일한 degradation 현상 재현. (MNIST에서도 비슷한 현상이 보고된 바 있어, 이게 특정 데이터셋만의 문제가 아니라 **근본적인 최적화 문제**라는 걸 시사함)
- **ResNets**: 반대로 층이 깊어질수록 training/test error가 계속 개선됨 $\rightarrow$ ImageNet과 마찬가지로 degradation 문제를 잘 해결하고, 깊이로부터 정확도 이득을 얻음

실험 2: 110층 ResNet ($n=18$)

- 초기 learning rate 0.1이 처음엔 너무 커서 수렴을 잘 시작하지 못함
- 그래서 warm-up 트릭 사용: 처음엔 learning rate를 0.01로 낮게 시작해서 training error가 80% 밑으로 내려갈 때까지(약 400 iteration) 학습한 뒤, 다시 0.1로 올려서 이어서 학습
- 결과: 파라미터 수(1.7M)는 FitNet, Highway 같은 기존 얇고 깊은 네트

method			error (%)
Maxout [10]			9.38
NIN [25]			8.81
DSN [24]			8.22
	# layers	# params	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72 $\pm$ 0.16)
Highway [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61 $\pm$ 0.16)
ResNet	1202	19.4M	7.93

Table 6. Classification error on the **CIFAR-10** test set. All methods are with data augmentation. For ResNet-110, we run it 5 times and show “best (mean $\pm$ std)” as in [43].

워크들보다 적은데도 6.43%라는
당시 SOTA급 test error 달성
(Table 6)

Analysis of Layer Responses

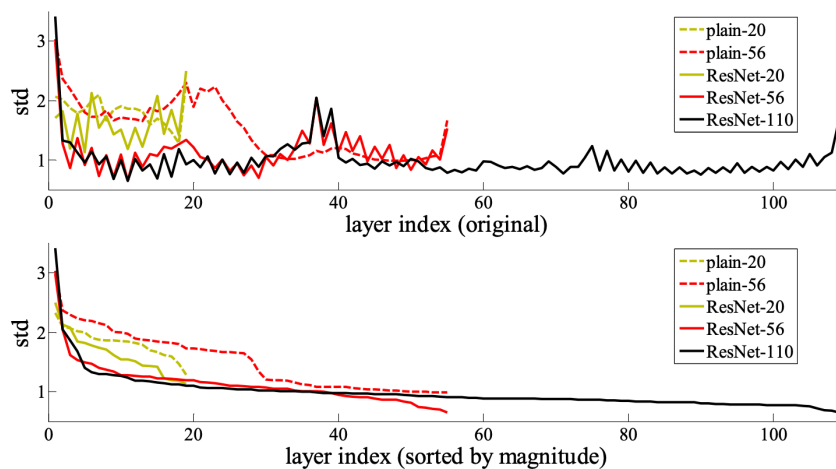
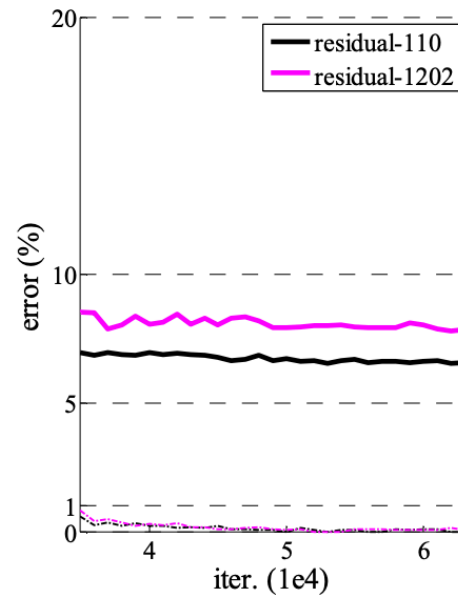


Figure 7. Standard deviations (std) of layer responses on CIFAR-10. The responses are the outputs of each 3×3 layer, after BN and before nonlinearity. **Top:** the layers are shown in their original order. **Bottom:** the responses are ranked in descending order.

- 각 3×3 conv layer의 출력(BN 이후, 비선형 함수 이전)의 표준편차를 측정
- ResNet의 layer response가 plain net보다 전반적으로 작음 → 이는 residual function이 대체로 0에 가깝게 학습된다는 논문의 핵심 motivation을 뒷받침
- 게다가 네트워크가 깊을수록(20 → 56 → 110층) 개별 layer의 response 크기가 더 작아짐 → layer 수가 많아질수록 각 layer가 신호를 아주 조금씩만 수정(fine adjustment)한다는 뜻이고, 이는 residual formulation이 의도한 대로 동작하고 있다는 증거

Exploring Over 1000 layers

- 이 정도로 극단적으로 깊어도 최적화 자체에는 문제가 없음 (training error가 0.1% 미만까지 내려감)
- test error도 7.93%로 나쁘지 않지만, 같은 training error를 가진 110층 네트워크보다는 오히려 test 성능이 더 나쁨
- overfitting 때문이라고 추정: 파라미터가 19.4M으로, CIFAR-10처럼 작은 데이터셋에는 지나치게 큰 모델일 수 있음
- 이 논문에서는 maxout이나 dropout 같은 강한 regularization을 따로 쓰지 않고, 단지 deep and thin 구조 설계 자체로만 규제를 걸었음 — optimization 문제에 집중하기 위해 일부러 다른 요소는 최소화



4.3. Object Detection on PASCAL and MS COCO

training data	07+12	07++12
test data	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

Table 7. Object detection mAP (%) on the PASCAL VOC 2007/2012 test sets using **baseline** Faster R-CNN. See also Table 10 and 11 for better results.

metric	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

Table 8. Object detection mAP (%) on the COCO validation set using **baseline** Faster R-CNN. See also Table 9 for better results.